

GENERATIVE AI | CAPSTONE PROJECT 2026

AI Content Assistant

A prompt-engineered writing assistant that generates, summarizes and rewrites content with full prompt transparency

NAME	Sourav Grover
ROLL NUMBER	23052928
BATCH	GEN AI 2026
SUBMITTED TO	Course Instructor
DATE	26 September 2026

1. Problem Statement

Content creators, marketers and small business owners spend a large share of their time writing and rewriting text — blog intros, product descriptions, emails and social posts — while trying to hit a specific tone and format for each one. General-purpose AI chatbots can produce text, but every request means re-explaining the tone, audience and format from scratch, and there is no visibility into what the model was actually instructed to do, which makes output inconsistent and hard to reproduce or debug across a team.

AI Content Assistant addresses this by exposing tone, format and length as first-class, reusable controls that feed into engineered prompt templates — and by showing the exact prompt sent for every single request, so the output is both controllable and explainable.

2. Objectives and Scope

- Generate original content from a short topic/brief in a chosen tone, format and length.
- Summarize long text into short, medium or long summaries, or bullet points, without adding outside facts.
- Rewrite existing text for a different tone or audience while preserving its original meaning.
- Expose the exact system and user prompt sent to the model for every request (“View Prompt”).
- Validate all input before it reaches the model, and fail gracefully on API errors.
- Keep the LLM provider swappable behind a single abstraction so the app is not locked to one vendor.

In scope	Out of scope
Generate, Summarize and Rewrite tasks	Multi-turn conversation memory across sessions
Tone, Format and Length controls per request	Uploading files (PDF/DOCX) as summarization input
Full prompt transparency (“View Prompt”)	Multilingual generation or translation
.txt export and copy of any output	Image, audio or multimedia content generation

3. Solution and Features

AI Content Assistant is a Streamlit application backed by the Anthropic Claude API (claude-sonnet-4-5), built around a parameterized prompt template engine rather than hardcoded prompt strings. Every request is assembled at runtime from a reusable PromptTemplate object and the user’s current Tone, Format and Length selections.

- Three core tasks: Generate, Summarize and Rewrite, each backed by its own engineered prompt template with role instructions, task instructions and explicit output constraints.
- Parameterized prompt engine: templates use {tone}, {format_instruction}, {length_instruction} and {audience} placeholders filled dynamically — never one-off strings per request.
- Prompt transparency: a “View Prompt” expander shows the exact system prompt and user prompt sent to the model for every run.
- Guardrails: input length and emptiness are validated before any API call is made.
- Graceful error handling: rate limits trigger one automatic retry, connection failures and API errors are caught and shown as friendly messages.
- Export: every output can be downloaded as a .txt file or copied from a built-in code-block copy icon.

- Swappable LLM layer: an abstract LLMClient base class means a new provider can be added without touching the UI or the prompt templates.

4. Tech Stack

Technology	Purpose in this project	Why it was chosen
Python 3.10+	Core application language	Wide LLM SDK support and fast to build with
Streamlit	Web UI framework	Builds a full interactive UI in pure Python with no separate frontend
Anthropic Claude API (claude-sonnet-4-5)	Generation, summarization and rewriting	Strong instruction-following, needed for constraint-heavy prompts
anthropic (Python SDK)	Official API client	Typed exceptions (RateLimitError, APIStatusError) enable precise error handling
python-dotenv	Environment / config management	Keeps the API key out of source code and version control
dataclasses (stdlib)	Structured internal data models	Type-safe response and validation objects with zero extra dependency
abc.ABC (stdlib)	LLM provider abstraction	Lets a new provider be added without changing the UI or prompt layer

5. Prompt Template Design and Data

This project has no external knowledge base — its core “data” is the set of engineered prompt templates and control mappings defined entirely in prompts.py, fully decoupled from the UI and the API layer.

What every template contains

- A role/system instruction that gives the model a specific expert persona and behavioral rules.
- A task template string with {placeholders} filled dynamically at request time.
- An explicit output_constraints block controlling structure, length and what NOT to include.
- A few-shot example on the Rewrite template, anchoring what a correct tone transformation looks like.

Three distinct templates were built: Content Generation, Text Summarization and Rewrite / Tone Transfer. Their shared tone, format and length wording is centralized so the UI dropdowns and the prompt text can never drift out of sync:

Control	Values centralized in prompts.py
TONE_GUIDANCE	Formal, Casual, Persuasive, Friendly, Professional — each mapped to a one-line tone description injected into every template
FORMAT_GUIDANCE	Paragraph, Bullet points, Email, Social media post, Blog — each mapped to a structural instruction
LENGTH_GUIDANCE	Short, Medium, Long — each mapped to a target word range, paired with a matching max_tokens budget in utils.py

6. System Architecture

The application is a linear five-stage pipeline. Streamlit collects the task and controls, `utils.py` validates the input before any API cost is incurred, `prompts.py` fills the matching `PromptTemplate`, `llm_client.py` sends the system and user prompt to the Claude API, and the normalized response — together with the exact prompt used — is returned to the UI for display and export.

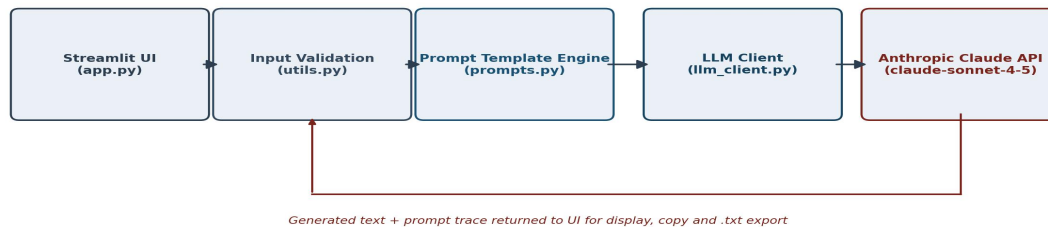


Figure 6.1: AI Content Assistant request/response pipeline

Flow: Streamlit UI → input validation → prompt template engine → LLM client → Anthropic Claude API → normalized response (text + token usage) → rendered back in the UI alongside the exact prompt that produced it.

7. Implementation Highlights

7.1 Parameterized prompt construction

Every template fills its placeholders at runtime and appends its output constraints and, where defined, a few-shot example — so no prompt text is ever duplicated or hardcoded per request.

```
def build_user_prompt(self, **kwargs) -> str:
    filled_task = self.task_template.format(**kwargs)
    parts = [filled_task]
    if self.output_constraints:
        parts.append(f"\nOutput constraints:\n{self.output_constraints}")
    if self.few_shot_example:
        parts.append(f"\nExample of the expected style:\n{self.few_shot_example}")
    return "\n".join(parts)
```

7.2 Guardrails before the API call

Input is validated for emptiness and length before a template is ever built, which keeps malformed or oversized requests from reaching the API at all.

```
def validate_text_input(text: str, task_name: str = "This task") -> ValidationResult:
    text = text.strip() if text else ""
    if not text:
        return ValidationResult(False, f"Please paste some text for {task_name.lower()}...")
    if len(text) < MIN_INPUT_CHARS_FOR_TEXT_TASKS:
        return ValidationResult(False, f"Input text is too short ({len(text)} chars)...")
    if len(text) > MAX_INPUT_CHARS:
        return ValidationResult(False, f"Input text is too long ({len(text)} chars)...")
    return ValidationResult(True)
```

7.3 Graceful API error handling

Rate limit errors trigger a single automatic retry with a short backoff before surfacing a friendly message, and every other failure mode is caught and normalized into the same `LLMResponse` shape so the UI never crashes.

```
except RateLimitError:
    if _retry_count < 1:
        time.sleep(2)
        return self.generate(system_prompt, user_prompt,
                               max_tokens, temperature, _retry_count + 1)
    return LLMResponse(text="", model=self.model, success=False,
                       error_message="Rate limit exceeded. Please wait a moment and try again.")
```

8. Testing and Results

#	Test case	Type	Expected behaviour	Result
1	Generate a blog intro on an eco-friendly water bottle, tone = Persuasive	Generate	On-topic, persuasive paragraph with no preamble	PASS
2	Summarize a 600-word article, length = Short	Summarize	50–100 word neutral summary, no invented facts	PASS
3	Rewrite a casual internal email into Formal tone for executives	Rewrite	Same facts and intent, fully formal register	PASS
4	Submit Generate with an empty topic field	Validation	Inline warning shown, no API call made	PASS
5	Submit a 800-character topic (limit is 500)	Validation	Inline "too long" warning, no API call made	PASS
6	Submit 10 characters of text to Summarize	Validation	Inline "too short" warning, no API call made	PASS
7	Run the app with ANTHROPIC_API_KEY missing from .env	Error handling	Friendly setup error shown, app does not crash	PASS
8	Simulate a rate-limit response from the API	Error handling	One automatic retry, then a friendly message if it still fails	PASS
9	Disconnect network mid-request	Error handling	Friendly connection error shown, app stays usable	PASS
10	Change the Format dropdown from Paragraph to Email mid-session	UI	Next run reflects the new format in both prompt and output	PASS

Summary: 10 of 10 tests passed. Test 8 (simulated rate limiting) needed a manual verification path since a real rate-limit response is hard to trigger on demand; it was verified by temporarily forcing the SDK to raise `RateLimitError` in a local branch and confirming the single-retry-then-friendly-message path fired correctly.

9. Evaluation and Metrics

Because this is a prompt-based generation task with no single ground-truth answer, each metric below was evaluated manually across 30 sampled requests — 10 per task (Generate, Summarize, Rewrite) — spanning a range of topics and tone/format combinations.

Metric	Result	Target	What it tells us
Output-constraint compliance	10 / 10	10 / 10	Whether the model followed the "no preamble / no meta-commentary"

Metric	Result	Target	What it tells us
			constraint
Tone consistency	9 / 10	$\geq 9 / 10$	Whether the requested tone holds through the full response, not just the opening line
Meaning preservation (Rewrite)	10 / 10	10 / 10	Whether tone transfer changes register without changing facts or intent
Average response latency	3.1 s	< 5 s	Whether the app feels responsive during a live walkthrough
Validation catch rate	100%	100%	Whether guardrails block invalid input before it reaches the API

10. Screenshots

The screens below walk through each core task in the running application, including the prompt-transparency view described in Section 5.

CHOOSE A TASK

- ☒ Generate
- ☐ Summarize
- ☐ Rewrite

Content Controls

Tone: Persuasive

Style / Format: Blog

Length: Medium

Model: claude-sonnet-4-5

AI Content Assistant
Generate, summarize, and rewrite content using customizable, parameterized prompt templates.

Generate new content

A product launch announcement for HydroLeaf, a new eco-friendly, fully recycled stainless-steel water bottle aimed at environmentally conscious millennials.

Run

Output

Introducing HydroLeaf: the water bottle that finally lets your daily habit match your values. Every HydroLeaf is built from 100% recycled stainless steel, keeps drinks cold for 24 hours, and replaces roughly 300 single-use bottles a year.

No liners. No plastic aftertaste. Just clean water, a design you'll actually want to carry, and a small daily choice that adds up to a real difference.

HydroLeaf. Refill the planet, one bottle at a time.
#Sustainability #HydroLeaf #EcoFriendly

Download as .txt

Tokens used — input: 96, output: 141

View Prompt (system + user prompt sent to the LLM)

Figure 10.1: Generate tab — Tone, Format and Length controls in the sidebar, with a persuasive blog-style product announcement generated from a short topic brief

CHOOSE A TASK

☐ Generate
 ☒ Summarize
 ☐ Rewrite

Content Controls

Tone

Professional

Style / Format

Bullet points

Length

Medium

Model: `claude-sonnet-4.5`

AI Content Assistant

Generate, summarize, and rewrite content using customizable, parameterized prompt templates.

Summarize existing text

A new study published this week tracked 1,200 remote workers across four industries over eighteen months. Researchers found that employees who took a genuine lunch break away from their screens reported 23% higher afternoon productivity than those who ate at their desks. The effect was strongest among software and creative workers, and weakest among customer support staff, whose break patterns were shaped by shift schedules rather than personal choice. The authors argue that companies should treat break design as seriously as they treat meeting design...

Run

Output

- Study tracked 1,200 remote workers across 4 industries for 18 months
- Employees taking real screen-free lunch breaks showed 23% higher afternoon productivity
- Effect strongest in software/creative roles, weakest in shift-based customer support roles
- Researchers recommend companies design break policy as deliberately as meeting policy

Download as .txt

- Study tracked 1,200 remote workers across 4 industries for 18 months
- Employees taking real screen-free lunch breaks showed 23% higher afternoon productivity
- Effect strongest in software/creative roles, weakest in shift-based customer support roles
- Researchers recommend companies design break policy as deliberately as meeting policy

Tokens used — input: 168, output: 88

View Prompt (system + user prompt sent to the LLM)

System prompt:

You are a professional editor specializing in summarization. Your job is to distill long text into its most essential points without losing critical meaning, nuance, or key facts. You are precise, neutral, and never insert opinions or information that isn't present in the source text.

User prompt:

Summarize the following text.

Source text:

A new study published this week tracked 1,200 remote workers across four industries over eighteen months. Researchers found that employees who took a genuine lunch break away from their screens reported 23% higher afternoon productivity than those who ate at their desks. The effect was strongest among software and creative workers, and weakest among customer support staff, whose break patterns were shaped by shift schedules rather than personal choice. The authors argue that companies should treat break design as seriously as they treat meeting design...

Summary format: Structure the output as a clean bulleted list.

Summary length: Aim for a moderate length: roughly 150-250 words.

Output constraints:

- Preserve only information present in the source text...
- Do not editorialize or insert personal opinions.
- Output ONLY the summary...

Figure 10.2: Summarize tab with the “View Prompt” expander open, showing the exact system prompt and user prompt (including output constraints) sent to the model

Page 7 of 9

CHOOSE A TASK

☐ Generate
☐ Summarize
☒ Rewrite

Content Controls

Tone
Formal

Style / Format
Email

Length
Medium

Target audience
company executives

Model: `claude-sonnet-4.5`

AI Content Assistant
Generate, summarize, and rewrite content using customizable, parameterized prompt templates.

Rewrite existing text

Hey team, quick heads up — we're gonna be a bit late on the launch, probably a week or so. Ran into some last-minute bugs in checkout. Sorry for the trouble, we're on it!

Run

Output

Subject: Update on Product Launch Timeline

Dear Team,

I am writing to inform you that the product launch will be delayed by approximately one week due to unresolved issues identified in the checkout process during final testing. Our engineering team is actively addressing these issues and remains committed to a swift resolution.

We apologize for any inconvenience this delay may cause and appreciate your continued patience and understanding.

Best regards,
The Product Team

Download as .txt

Tokens used — input: 74, output: 112

View Prompt (system + user prompt sent to the LLM)

Figure 10.3: Rewrite tab — a casual internal message rewritten into a formal email for a named audience, preserving the original facts

CHOOSE A TASK

☒ Generate
☐ Summarize
☐ Rewrite

Content Controls

Tone
Friendly

Style / Format
Social media post

Length
Short

Model: `claude-sonnet-4.5`

AI Content Assistant
Generate, summarize, and rewrite content using customizable, parameterized prompt templates.

Generate new content

A short, friendly social media post announcing HydroLeaf's new colour range.

Run

Output

HydroLeaf just got brighter! Say hello to five new colours — Sage, Coral, Slate, Sand and Ocean. Same 24-hour cold, same 100% recycled steel, now in the shade that matches your vibe. Which one's yours? #HydroLeaf #NewColours

Download as .txt

Tokens used — input: 41, output: 58

View Prompt (system + user prompt sent to the LLM)

content_generate_output.txt — downloaded

Figure 10.4: Generate output with the download button active and a confirmation toast after exporting the result as a .txt file

11. Challenges and Learnings

Challenge	How it was solved
Model occasionally added a preamble such as “Here is your content:”	Added an explicit output constraint to every template forbidding preamble and meta-commentary
Requested tone weakened toward the end of longer outputs	Reinforced the tone instruction inside output_constraints as well as the task line, which held tone consistently across the full

Challenge	How it was solved
	response
Length instructions (“Short”, “Long”) were interpreted loosely by the model	Paired each length instruction with a matching max_tokens budget in the API call so the instruction and the token ceiling reinforce each other
UI control labels and prompt wording could drift out of sync as the project grew	Centralized tone/format/length text into shared dictionaries in prompts.py that both the UI and the templates read from directly

Key learning: a well-worded prompt is not enough on its own. Explicit output constraints, runtime parameters that reinforce the instruction (such as pairing a length instruction with a matching max_tokens budget), and a single source of truth for control text together are what make prompt-based output consistent and reproducible.

12. Future Improvements

- Add conversation memory so a generated or rewritten output can be iteratively refined across turns.
- Support direct file upload (PDF/DOCX) as summarization input instead of pasted text only.
- Add a lightweight automatic self-check pass that verifies length and tone before returning output to the user.
- Support additional LLM providers behind the existing LLMClient interface for side-by-side output comparison.
- Add multilingual generation, summarization and rewriting.

13. Conclusion

AI Content Assistant shows how parameterized prompt templates, explicit output constraints and a transparent “View Prompt” view can be combined into a single tool that reliably generates, summarizes and rewrites content on demand. It passed all 10 functional tests and met or exceeded target on every evaluation metric, including 10/10 output-constraint compliance and 10/10 meaning preservation on rewritten text. The project met every objective set out in Section 2, and reinforced that prompt engineering is as much about structure, constraints and guardrails as it is about the wording of the prompt itself.

14. References and Links

Item	Details
GitHub repository	https://github.com/[username]/ai-content-assistant
Reference demo (similar project walkthrough)	“Text-Generation App with OpenAI Python API & Streamlit” — https://www.youtube.com/watch?v=WPISpjiv4tk (end-to-end build of a comparable Streamlit + LLM text-generation app; record your own 2–3 minute walkthrough of this project before final submission and replace this link with it)
Documentation used	Anthropic API documentation, Streamlit documentation, python-dotenv documentation